

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет
"Высшая школа экономики"»

Московский институт электроники и математики им.
А.Н. Тихонова НИУ ВШЭ
Департамент компьютерной инженерии

Курс: Алгоритмизация и программирование

ОТЧЕТ
по лабораторной работе №5

Раздел	Максимальная оценка	Итоговая оценка
Работа программы	1	
Тесты	1	
Правильность алгоритма	3	
Листинг	0,5	
Ответы на вопросы	2	
Дополнительное задание	3	
Итого		

Студент: Солодилов Михаил Анатольевич
Группа: БИВ253
Вариант: 232 (5, 12, 2)
Руководитель:
Оценка:
Дата сдачи:

Оглавление

Задание	2
Внешняя спецификация	3
Листинг программы	4
Распечатка тестов к программе и результатов	8

Задание

1. Создать файл для хранения действительных чисел, вводимых с клавиатуры. Прочитать этот файл и вычислить максимальное среди отрицательных чисел.
2. Создать связанный список для хранения целых чисел. Число записей неизвестно, но можно ввести число, являющееся признаком окончания ввода данных. Для четных номеров вариантов организовать очередь, а для нечетных – стек. Для исходного списка решить следующую задачу: вставить заданное число $A1$ после каждого элемента с нечетным номером.
3. Для полученного списка решить следующую задачу: упорядочить по убыванию методом «пузырька» элементы списка, расположенные после первого положительного элемента.

Внешняя спецификация

Лабораторная работа №5

Задание №1

Введите действительные числа, ввод закончите пустой строкой:
< x >

До пустой строки
При $max = -\infty$

В последовательности не было отрицательных чисел

Иначе

$max = \ll max \gg$

Задание №2

Введите целые числа, ввод закончите пустой строкой:
< n >

До пустой строки

Очередь:
 $\ll q[0] \gg, \ll q[1] \gg, \dots$

Введите A1:
< $A1$ >

Очередь:
 $\ll q[0] \gg, \ll q[1] \gg, \dots$

Задание №3

При $pos = NULL$

Нет положительных элементов

Иначе

Очередь:
 $\ll q[0] \gg, \ll q[1] \gg, \dots$

Листинг программы

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <stdlib.h>
4
5 typedef struct Node
6 {
7     int data;
8     struct Node* next;
9 } Node;
10
11 typedef struct Queue
12 {
13     Node* head;
14     Node* tail;
15 } Queue;
16
17 static void push(Queue* q, int n);
18 static void insert(Node* after, int n);
19 static void queue_dtor(Queue* queue);
20 static void print_queue(const Queue queue);
21 static Queue input_queue(void);
22 static void insert_after_odd(Queue* queue, int a1);
23
24 static Node* find_positive(Queue queue);
25 static void sort(Node* node);
26
27 static void clean_buffer(FILE* stream);
28
29 static void read_stdin(const char* save_path);
30 static double find_max_neg(const char* path);
31
32 int main()
33 {
34     puts("Лабораторная работа №5");
35
36     puts("Задание №1");
37
38     read_stdin("data.txt");
39
40     double max = find_max_neg("data.txt");
41
42     if (max == -INFINITY)
43     {
44         puts("В последовательности не было отрицательных чисел");
45     }
46     else
47     {
48         printf("max = %lg\n", max);
49     }
50
51     puts("Задание №2");
52
53     Queue q = input_queue();
54     puts("Очередь:");
55     print_queue(q);
56
57     int a1 = 0;
58     puts("Введите A1:");
59     scanf("%d", &a1);
60     insert_after_odd(&q, a1);
61     puts("Очередь:");
62     print_queue(q);
63
64     puts("Задание №3");
65     Node* pos = find_positive(q);
66     if (!pos)
67     {
68         puts("Нет положительных элементов");
69     }
70     else
71     {
72         sort(pos);
73         puts("Очередь:");
```

```

74     print_queue(q);
75 }
76
77 queue_dtor(&q);
78 }
79
80 static void push(Queue* q, int n)
81 {
82     if (q->tail == NULL)
83     {
84         q->tail = calloc(1, sizeof(Node));
85         q->tail->data = n;
86         q->head = q->tail;
87         return;
88     }
89
90     Node* new_node = calloc(1, sizeof(Node));
91     new_node->data = n;
92
93     q->tail->next = new_node;
94     q->tail = new_node;
95 }
96
97 static void insert(Node* after, int n)
98 {
99     Node* new_node = calloc(1, sizeof(Node));
100     new_node->data = n;
101     new_node->next = after->next;
102     after->next = new_node;
103 }
104
105 static void queue_dtor(Queue* queue)
106 {
107     Node* cur = queue->head;
108
109     while (cur)
110     {
111         Node* next = cur->next;
112         free(cur);
113         cur = next;
114     }
115
116     *queue = (Queue){};
117 }
118
119 static void print_queue(const Queue queue)
120 {
121     Node* cur = queue.head;
122
123     while (cur)
124     {
125         Node* next = cur->next;
126         printf("%d\n", cur->data);
127         cur = next;
128     }
129 }
130
131 static Queue input_queue(void)
132 {
133     Queue q = {};
134
135     char buffer[512] = "";
136
137     puts("Введите целые числа, ввод закончите пустой строкой:");
138
139     fgets(buffer, sizeof(buffer), stdin);
140     while (buffer[1] != '\0')
141     {
142         int n = 0;
143         sscanf(buffer, "%d", &n);
144         push(&q, n);
145         fgets(buffer, sizeof(buffer), stdin);
146     }
147
148     return q;
149 }

```

```

150
151 static void insert_after_odd(Queue* queue, int a1)
152 {
153     size_t n = 1;
154     Node* cur = queue->head;
155
156     while (cur)
157     {
158         Node* next = cur->next;
159         if (n % 2 == 1)
160         {
161             insert(cur, a1);
162         }
163         cur = next;
164         n++;
165     }
166 }
167
168 static Node* find_positive(Queue queue)
169 {
170     Node* cur = queue.head;
171
172     while (cur)
173     {
174         if (cur->data > 0)
175         {
176             return cur;
177         }
178         cur = cur->next;
179     }
180
181     return NULL;
182 }
183
184 static void sort(Node* node)
185 {
186     if (!node)
187     {
188         return;
189     }
190
191     Node* n1 = node;
192
193     while (n1)
194     {
195         Node* n2 = n1->next;
196         while (n2)
197         {
198             if (n1->data < n2->data)
199             {
200                 int t = n2->data;
201                 n2->data = n1->data;
202                 n1->data = t;
203             }
204             n2 = n2->next;
205         }
206         n1 = n1->next;
207     }
208 }
209
210 static void read_stdin(const char* save_path)
211 {
212     FILE* save = fopen(save_path, "w");
213
214     char buffer[512] = "";
215
216     puts("Введите действительные числа, ввод закончите пустой строкой:");
217
218     fgets(buffer, sizeof(buffer), stdin);
219     while (buffer[1] != '\0')
220     {
221         double n = 0;
222         sscanf(buffer, "%lg", &n);
223         fprintf(save, "%lg\n", n);
224         fgets(buffer, sizeof(buffer), stdin);
225     }

```

```

226     fclose(save);
227 }
228
229
230 static double find_max_neg(const char* path)
231 {
232     FILE* f = fopen(path, "r");
233
234     char buffer[512] = "";
235
236     double max = -INFINITY;
237
238     while (fgets(buffer, sizeof(buffer), f))
239     {
240         double n = 0;
241         sscanf(buffer, "%lg", &n);
242
243         if (n < 0 && n > max)
244         {
245             max = n;
246         }
247     }
248
249     fclose(f);
250
251     return max;
252 }
253
254 static void clean_buffer(FILE* stream)
255 {
256     while (fgetc(stream) != '\n');
257 }

```

Распечатка тестов к программе и результатов

№	Исходные данные	Результат
1	$k = 1, S[0 : 0] = [\text{abba}]$	Не нашлось подстрок, ограниченных точками Не нашлось строки со скобками и цифрами
2	$k = 2, S[0 : 2] = [\text{y..b..c ..d, .dog при(1-2)ветfe.}]$	Нашлось 3 подстрок, ограниченных точками: b c dog при(1-2)ветfe Нашлась строка со скобками и цифрами: "dog при(1-2)ветfe" После очистки букв не из русского алфавита: "привет"
3	$k = 1, S[0 : 0] = [\text{.adas.dog приветfeline.}]$	adas dog приветfeline Не нашлось строки со скобками и цифрами